

Chapter 6

Optimal Decomposition for Distributed Optimization in Nonlinear MPC by Community Detection

6.1 Introduction

Large-scale and complex systems are becoming the rule rather than the exception in process systems engineering problems [135, 18]. The design and operation of these systems involve large-size, non-convex, mixed-integer, multi-scale and uncertain optimization models [113, 270, 250, 136, 122] which are computationally expensive to solve. Decomposition is a major approach to solving these large-scale optimization problems. To deal with the complicating variables and constraints between different groups of unit operations, Benders and Lagrangian decompositions have been recognized as the most useful decomposition methods [66, 136]. Computational complexity is also a major issue in the implementation of many advanced closed-loop decision making schemes due to the requirement of real-time repeated solution of optimization problems. Examples include nonlinear model predictive control (NMPC) [344, 254], economic MPC (EMPC) [343, 98], closed-loop scheduling [142, 317] as well as their integration [19, 322, 86].

NMPC is an advanced control strategy where the manipulated inputs of a nonlinear process are determined by minimizing a cost associated with the predicted trajectory of the system based on real-time measurements. During the recent decade, highly efficient nonlinear programming algorithms, toolboxes (e.g., IPOPT [437, 37] and GPOPS [340]) and modelling platforms (e.g., CasADi [8], JuMP [93] and Pyomo [147]) have been developed to enable fast implementation of NMPC. Under interior-point optimization algorithms, a decomposition at the linear algebra level can be exploited for further acceleration by parallelizing the solution of the Karush-Kuhn-Tucker (KKT) conditions, if there exists an approximate block diagonal [67], bordered block diagonal [477, 187], or block tridiagonal [440] structure in the KKT matrix. Other techniques such as truncated Newtonian optimization [87, 474] and advanced-step optimization [476, 461] have also been proposed for the fast solution of optimal control problems.

©2018 Elsevier. Reprinted, with permission, from W. Tang, A. Allman, D. Babaei Pourkargar, P. Daoutidis. Optimal decomposition for distributed optimization in nonlinear model predictive control through community detection. *Computers & Chemical Engineering* (Volume 111, March 2018). DOI: 10.1016/j.compchemeng.2017.12.010.

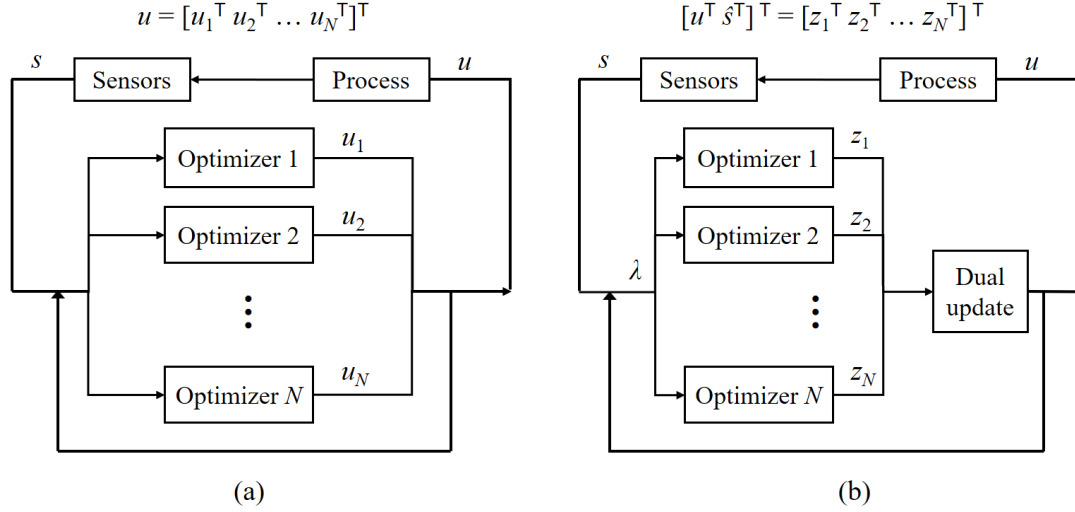


Figure 6.1: Two formulations and distributed optimization methods of the optimal control problem in NMPC. (a) BCD algorithms for the “input problem” formulation. (b) ADMM algorithms for the “input-state problem” formulation. u , s , $\hat{s}$, z , and λ stand for inputs, states, predicted states, primal variables and dual variables, respectively.

6.1.1 Distributed optimization for NMPC

Distributed optimization formulations of the associated optimal control problems, referred to as distributed model predictive control, have been a major focus recently (see [362, 63, 288] for reviews and references) due to fewer communication requirements, resilience to faults, and the potential of further improving the computational performance. So far, two types of distributed optimization algorithms – block coordinate descent (BCD) algorithms [452, 285] and multi-block alternative direction method of multipliers (ADMM) [44, 229, 162], have been applied in distributed NMPC.

In most of these works, the optimal control problem is formulated such that the manipulated inputs are the variables to be optimized, and the state and output variables are explicitly expressed in terms of inputs (see e.g., [234]). We refer to this as an “*input problem*” formulation. By BCD (see Fig. 6.1(a)), the input variables in the system are assigned to different controllers, and each controller solves an optimization problem for its own inputs. The controllers exchange their local solutions in a sequential, iterative or neighbor-to-neighbor scheme. We note that the BCD algorithms under an “input problem” formulation, in the presence of complex constraints, can cause the solution trajectory to escape from the feasible set if the optimization subproblems are solved in

parallel, unless the feasible set is box-shaped, i.e., the constraints on the manipulated inputs include only individual bounds. Further, when the objective function is a complex function of the manipulated inputs, it becomes difficult to guarantee the optimality of the solution obtained.

In a different vein, the optimal control problem can also be formulated such that the inputs and the predicted states are both variables to be optimized, and these variables are (nonlinearly) constrained by the system dynamical equations (see e.g., [476]). In this way, the objective function is expressed in a simpler form (usually a quadratic form in terms of the inputs and the states) while the constraints become more complex. We refer to this as an “*input-state problem*” formulation. By ADMM (see Fig. 6.1(b)), an augmented Lagrangian is constructed to deal with the complex constraints, and the primal variables, namely the variables to be optimized, are updated through optimization in groups, with the dual variables updated after the optimization of primal variables [332, 166, 167]. The Lagrangian-based algorithms under an “input-state problem” formulation can potentially avoid the limitations of the “input problem” formulation. Through a dualization of the constraints, the solution trajectory is no longer confined inside the feasible set. With a simpler form of the objective function, the optimality of the solution obtained can be improved. On the other hand, the “input-state problem” involves a higher-dimensional optimization, and thus may generally require a longer computational time than the “input problem”.

6.1.2 Choice of optimal decomposition

To implement any distributed control or optimization strategy, a decomposition, from which good control performance as well as reduced computational time can be expected, must be predetermined. The topic of system decomposition has been studied in different contexts for a long time. For dynamic systems and control systems, decompositions were proposed for decentralized control design [434, 321, 372]. In process system engineering, algorithms for tearing process flowsheets were proposed for process simulation and synthesis (see e.g., [24, 320, 426]). The decomposition of systems of equations was studied even earlier (see e.g., [92, 396, 156]). The key of these decomposition methods is to detect the underlying structures in the system – most importantly, blocks and hierarchies – with graph-theoretic representations and tools.

Network science has recently emerged as a promising framework for the analysis of complex systems based on macroscopic and statistical properties of graphs (networks)

[21]. In this setting, *community detection* has become a major field of study. Different from the traditional decomposition methods that seek a strict block diagonal or block triangular structure using paths, cycles or coverings, community detection aims to find subnetworks with significantly more links between the nodes inside than across them [127]. The decomposition can be efficiently generated for large and complex networks, and is optimal in the sense of the difference between inter- and intra-community interconnections, which is desirable for distributed control and optimization in networks. Details of this approach will be introduced later in Section 6.2.

Recently in our research, we have developed some methods of input-output partitioning for distributed control [180, 406, 408]. These decomposition methods, based on the analyses of the interactions among the process variables (inputs, states and outputs), generate compact and weakly interacting subsystems through community detection. It was shown that the choice of decomposition is indeed a major factor affecting the control performance and computational time of the resulting control scheme, and that the adoption of appropriate decompositions can relieve the computational burden while maintaining good control performance [326, 327]. However, these input-output partitioning methods were not specifically designed for NMPC applications, and were applicable only to “input problem” formulations. To find an optimal decomposition for the “input-state problem” formulation, we need to address the problem of *optimally decomposing the optimization problems*.

So far, decomposition for distributed optimization is typically done by intuitively finding such complicating variables and complicating constraints that the optimization problem becomes completely separated once these complicating variables or constraints are removed. For general problems without a special structure, there has been no systematic method for finding an optimal decomposition for distributed optimization.

In this work, we introduce network representations of a broad class of optimization problems, and propose an efficient framework of determining the optimal decomposition of these optimization problems based on community detection in these network representations. This framework is based on the structure of the optimization problem rather than on the specific parameters (namely the measurements at different sampling times). It can therefore be applied offline to generate a decomposition in advance of the online implementation of NMPC.

The remainder of this chapter is organized as follows. In Section 6.2, we first introduce the notion of variable-constraint bipartite network, variable unipartite network,

and constraint unipartite network as structural representations of optimization problems with an objective function in separable form. Through the study of a large-scale convex optimization problem, we show the computational benefit of decomposing the network into community structures which are strongly interconnected inside but weakly interconnected between. Community detection algorithms as powerful tools in network science for finding modular structures are subsequently introduced. In Section 6.3, the network community detection framework is applied to generate the optimal decomposition of the optimal control problems in NMPC. In Section 6.4, we examine a benchmark reactor-separator process with comparisons of different decompositions under an ADMM algorithm slightly modified from that of [166] and the IPOPT algorithm. We conclude that the community detection in the variable unipartite network and the variable-constraint bipartite network result in the optimal decompositions, respectively.

6.2 Community Detection for the Decomposition of Optimization problems

6.2.1 Network representations of optimization problems

We consider without loss of generality a constrained optimization problem in the following “separable” form:

$$\begin{aligned} \min \quad & f_1(v_1) + f_2(v_2) + \cdots + f_n(v_n) \\ \text{s.t.} \quad & c_j(v_1, v_2, \dots, v_n) = 0, \quad j = 1, 2, \dots, m \\ & v_i \in \mathcal{V}_i, \quad i = 1, 2, \dots, n \end{aligned} \tag{6.1}$$

where $v_i, i = 1, 2, \dots, n$ are scalar variables, and $\mathcal{V}_i, i = 1, 2, \dots, n$ represents the set (interval) to which the variables belong. Note that in (6.1), the objective function is separable and the variables are coupled only by equality constraints (with every inequality constraint only acting upon a single variable). If a non-separable objective function or coupling inequality constraints are present, the original optimization problem can always be converted to the form of (6.1) by defining suitable new auxiliary variables and constraints. To decompose the optimization problem into a distributed architecture, we seek to exploit the inherent structure of the optimization formulation, i.e., the *interactions* between the variables $v_1, v_2, \dots, v_n$ and the constraints $c_1, c_2, \dots, c_m$.

The structural information of the variable-constraint interactions can be represented

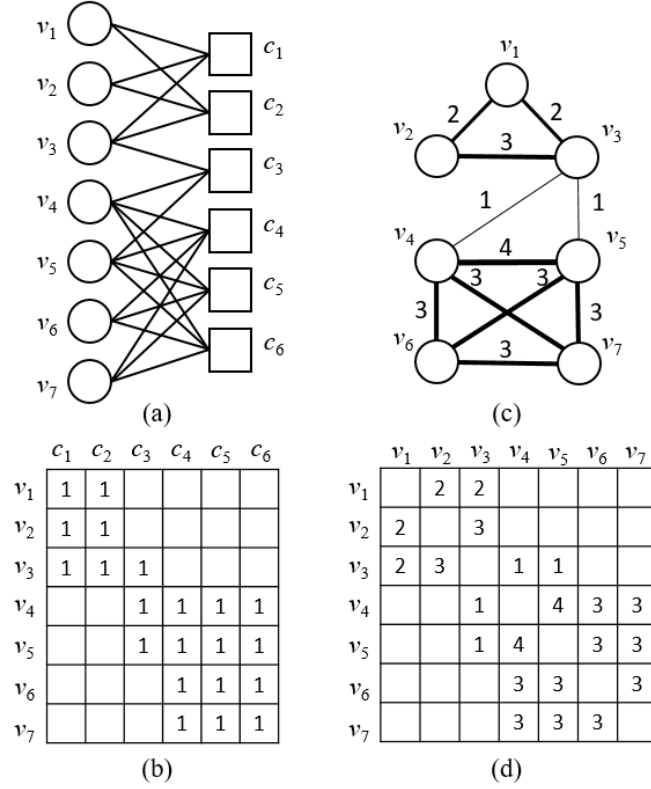


Figure 6.2: Network representations of optimization problems. (a) Variable-constraint bipartite network. (b) Bipartite adjacency matrix. (c) Variable unipartite network. (d) Edge weight matrix of the variable unipartite network.

by constructing a *variable-constraint bipartite network (graph)* (see Fig. 6.2(a)). In this bipartite network, the nodes are classified into two sets – the set of variables $\{v_1, v_2, \dots, v_n\}$ and the set of constraints $\{c_1, c_2, \dots, c_m\}$, and every edge connects a variable node and a constraint node: if the variable v_i is present in the expression of c_j (i.e., $\partial c_j / \partial v_i \neq 0$, assuming the continuous differentiability of all the constraints), then there is an edge between v_i and c_j . The network information can be equivalently expressed by a bipartite adjacency matrix $\mathbf{B} \in \{0, 1\}^{n \times m}$, where its (i, j) -th element $b_{ij} = 1$ if an edge exists between v_i and c_j , and $b_{ij} = 0$ otherwise (see Fig. 6.2(b)). The bipartite adjacency matrix $\mathbf{B}$ is simply the transposed Jacobian matrix $[\partial c_j / \partial v_i]$, with every nonzero element substituted by 1.

The information in the bipartite network can be simplified into a *variable unipartite network* which captures the interactions between the variables (see Fig. 6.2(c)). The

nodes in the variable unipartite graph include only the variables $v_1, v_2, \dots, v_n$. The weight of the edge between v_i and $v_{i'}$, $u_{ii'}$, is defined as the number of the constraints to which both v_i and $v_{i'}$ are adjacent in the variable-constraint bipartite network, i.e.,

$$u_{ii'} = \sum_{j=1}^m b_{ij} b_{i'j} = \left(\mathbf{B} \mathbf{B}^\top \right)_{ii'}. \quad (6.2)$$

The variable unipartite network can also be represented by the edge weight matrix $\mathbf{U} = [u_{ii'}] \in \mathbb{N}^{n \times n}$ (see Fig. 6.2(d)). In graph theory, the variable unipartite network is said to be the one-mode projection of the variable-constraint network onto the variable set [487]. Compared to the bipartite network, for each pair of variables, the unipartite network only tells *how many* constraints they share, without providing information on *which* constraints they share.

We can also define the *constraint unipartite network* to capture the interactions between the constraints in the same way as for the unipartite unipartite network. The corresponding edge weight matrix $\mathbf{W} = [w_{jj'}] \in \mathbb{N}^{m \times m}$ is obtained by

$$w_{jj'} = \sum_{i=1}^n b_{ij} b_{ij'} = \left(\mathbf{B}^\top \mathbf{B} \right)_{jj'}. \quad (6.3)$$

To decompose the optimization problem, we propose to partition the variables (constraints) into groups such that the interactions in the same groups, i.e., the common involvement in the constraints (variables) between pairs of variables (constraints) in the same groups, are stronger than the interactions between a pair of variables (constraints) across different groups. In network theory terminology, such groups are called *community structures*, and the procedure to reveal the communities in networks is called *community detection* [127]. Community structures are widely present in biological, social and technological networks, and community detection has already become a major area of study in network science [114, 294, 115]. Illustrations of community structures in bipartite and unipartite networks are given in Fig. 6.3(a) and (b), respectively.

Community structures can not usually be determined intuitively, and therefore efficient community detection algorithms should be adopted. Before introducing such algorithms, we consider a typical large-scale convex ℓ_1 -norm minimization problem with highly coupling equality constraints to show how the different decompositions of the

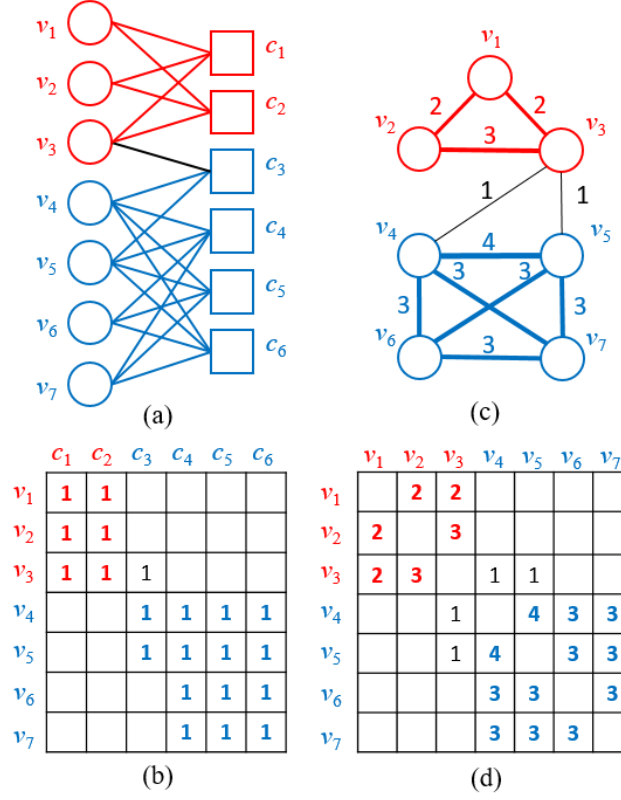


Figure 6.3: Communities in variable-constraint bipartite network (a,b) and variable unipartite network (c,d). The edge weights between communities formed by v_1, v_2, v_3 (red) and v_4, v_5, v_6, v_7 (blue) is smaller than the edge weights inside the communities. The communities are represented in both the network topology (a,c) and the adjacency matrix (b,d).

same optimization problem make a difference in the solution, in the presence of communities.

6.2.2 The role of community structures in the decomposition of optimization problems

We consider a quadratic minimization subject to linear constraints, also known as the “basis pursuit problem” [44]:

$$\min_v \|v\|_1 = \sum_{i=1}^n |v_i| \quad \text{s.t.} \quad Av = b \quad (6.4)$$

where $A \in \mathbb{R}^{m \times n}$, $b \in \mathbb{R}^{n \times 1}$. This model can be taken as an optimization problem associated with the unconstrained MPC of a linear system if the cost function assumes the form of an ℓ_1 -norm. We solve the optimization problem in a distributed manner by dividing the variables into groups $v^1, v^2, \dots, v^N$, $v^i = [v_1^i, v_2^i, \dots, v_{n_i}^i]^\top$, $i = 1, 2, \dots, N$, and using the parallel multi-block ADMM method [44]. Specifically, an augmented Lagrangian is constructed, in which λ is the Lagrange multiplier, γ is a penalty coefficient enforcing the constraints, and A^i is composed of the columns of A corresponding to variable block v^i :

$$\begin{aligned} L_\gamma(v^1, v^2, \dots, v^N; \lambda) &= \|v\|_1 - \lambda^\top (Av - b) + \frac{\gamma}{2} \|Av - b\|_2^2 \\ &= \sum_{i=1}^N \|v^i\|_1 - \lambda^\top \left(\sum_{i=1}^N A^i v^i - b \right) + \frac{\gamma}{2} \left\| \sum_{i=1}^N A^i v^i - b \right\|_2^2. \end{aligned} \quad (6.5)$$

The minimizations are performed on primal variables in parallel blocks, followed by a dual variable update in each iteration:

$$\begin{aligned} v^1 &:= \arg \min_{z^1} L_\gamma(z^1, v^2, \dots, v^N; \lambda) \\ v^2 &:= \arg \min_{z^2} L_\gamma(v^1, z^2, \dots, v^N; \lambda) \\ &\dots \\ v^N &:= \arg \min_{z^N} L_\gamma(v^1, v^2, \dots, z^N; \lambda) \\ \lambda &:= \lambda - \gamma \left(\sum_{i=1}^N A^i v^i - b \right) \end{aligned} \quad (6.6)$$

To examine the effect of decompositions on the computational performance, we generate a matrix A of size 1000×600 with nonzero elements more densely located within 4 separate blocks, which is visualized in Fig. 6.4. These 4 blocks which appear almost yellow in Fig. 6.4 correspond to 4 communities of variables. The vector b is generated with the same procedure as in [229] by $b = Av^* + w$ where v^* has 50 nonzero components randomly generated in $[-1, 1]$, and w is a random noise vector with identically distributed Gaussian components, each with a zero mean and a standard deviation of 10^{-3} .

We consider five different decompositions of the variables: (i) Decomposition according to the community structures, i.e., $v^1 = v_{1:200}$, $v^2 = v_{201:400}$, $v^3 = v_{401:650}$,

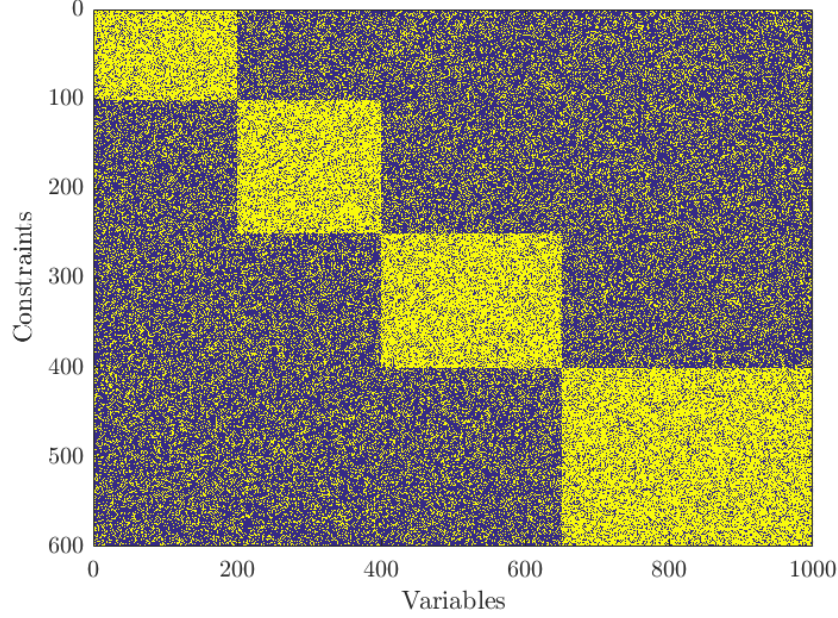


Figure 6.4: Visualization of the constraint matrix A in the ℓ_1 -norm minimization problem. Each nonzero element is represented a yellow pixel and a zero element by a blue pixel.

$v^4 = v_{651:1000}$. (ii) Decomposition with equal number of variables $v^1 = v_{1:250}$, $v^2 = v_{251:500}$, $v^3 = v_{501:750}$, $v^4 = v_{751:1000}$. (iii) Decomposition in randomized four groups of indices, with the size of the groups equal to those of the communities. (iv) A decomposition coarser than (i): $v^1 = v_{1:400}$, $v^2 = v_{401:1000}$. (v) A decomposition finer than (i): $v^1 = v_{1:200}$, $v^2 = v_{201:400}$, $v^3 = v_{401:525}$, $v^4 = v_{526:650}$, $v^5 = v_{651:825}$, $v^6 = v_{826:1000}$. As the iterations proceed, we record the relative error defined by $\|v - v_{\text{opt}}\|_2 / \|v_{\text{opt}}\|_2$, in which the v_{opt} is the true optimal solution obtained by directly solving the centralized optimization problem. The variations of the relative error with increasing iteration numbers are shown in Fig. 6.5 for four decompositions, except for the randomized decomposition which does not result in a solution trajectory convergent to the optimal solution v^* with increasing iterations.

Remark 6.1. For the constrained ℓ_1 -norm minimization problem, each block minimization step in (6.6) is performed in the following way. By defining an auxiliary variable $\epsilon \in \mathbb{R}^m$,

$$A^i v^i + \sum_{j \neq i} A^j v^j - b + \epsilon = \frac{1}{\gamma} \lambda, \quad (6.7)$$

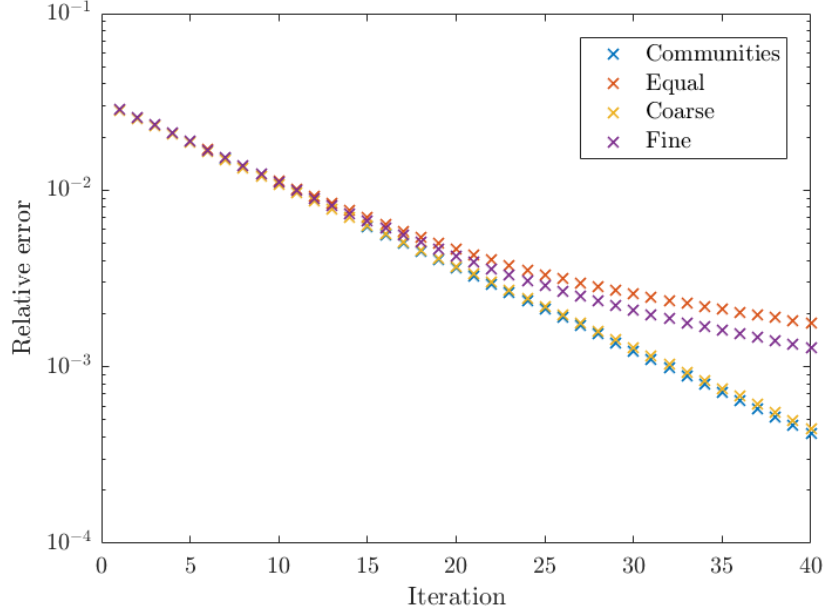


Figure 6.5: Convergence rate of the multi-block ADMM algorithm applied to the ℓ_1 -norm minimization problem using different decompositions.

the subproblem of optimizing v^i can be transformed to the following form

$$\min_{v^i, \epsilon} \|v^i\|_1 + \frac{\gamma}{2} \epsilon^\top \epsilon. \quad (6.8)$$

Denote by v_+^i and v_-^i the positive and the negative part of v^i , respectively, i.e., $v_+^i = (|v^i| + v^i)/2$ and $v_-^i = (|v^i| - v^i)/2$. Denote by $\mathbf{1}$ the column vector of 1. The v^i optimization step is hence transformed into a quadratic programming problem.

$$\begin{aligned} \min_{v_+^i, v_-^i, \epsilon} \quad & \mathbf{1}^\top v_+^i + \mathbf{1}^\top v_-^i + \frac{\gamma}{2} \epsilon^\top \epsilon \\ \text{s.t.} \quad & A^i v_+^i - A^i v_-^i + \epsilon = \frac{1}{\gamma} \lambda + b - \sum_{j \neq i} A^j v^j, \quad v_+^i \geq 0, \quad v_-^i \geq 0. \end{aligned} \quad (6.9)$$

To obtain the true optimal value v^* , the centralized optimization is converted to the following linear programming problem, which can be exactly solved, by defining the positive and negative parts of v , $v_+ = (|v| + v)/2$, $v_- = (|v| - v)/2$.

$$\min_{x_+, x_-} \quad \mathbf{1}^\top v_+ + \mathbf{1}^\top v_- \quad \text{s.t.} \quad A v_+ - A v_- = b, \quad v_+ \geq 0, \quad v_- \geq 0. \quad (6.10)$$

Remark 6.2. We choose this particular example in the form of (6.4) for the following reasons: (i) The ℓ_1 -norm minimization problem is convex, which has a unique optimum, thus facilitating the comparison. (ii) The problem is not strongly convex, thus allowing distributed ADMM to diverge under some bad decompositions. (iii) This problem is often used in the literature of distributed optimization to examine the performance of algorithms.

From Fig. 6.5, we can make the following observations:

1. Among the five decompositions above, the randomized decomposition results in divergence, i.e., the distributed optimization can lose its basic performance when the community structure is totally not accounted for in the decomposition. The reason for this is that a random decomposition will generally result in strong interactions between variable groups. Therefore, as long as the problem is not strongly convex (such as the weakly convex ℓ_1 -norm minimization), the decomposition for distributed optimization should not be randomly generated.
2. The decomposition of the variables into 4 groups of equal sizes, as a decomposition that possesses a certain degree of similarity to the community structures in the network, leads to convergence.
3. The convergence is accelerated by using a decomposition according to the communities, which renders the difference between cross-group interactions and within-group interactions significant. The coarser decomposition, where each group in the decomposition can contain more communities, has about the same performance as the decomposition exactly based on communities. Thus, *community structures act as the “core” playing a central role in maintaining good performance in the decomposition for distribution optimization.*
4. The finer decomposition, in which each group of variables only contains a portion of the corresponding community, deteriorates the convergence rate. This implies that the convergence performance is correlated to the integrity of the community structures.

Compared to the community-based decomposition, the coarser decompositions merge weakly interacting variable groups (communities) into larger groups. Except for very fast problems, larger subproblems will generally increase the computational burden of

each iteration. Since the interactions are already concentrated in the communities, accounting for inter-community interactions does not significantly improve the optimality of the solutions.

The above observations and arguments make a strong case for using a community-based decomposition for distributed optimization. In the following subsection, we introduce the computationally efficient *Louvain algorithm (fast unfolding algorithm)* of community detection [39].

6.2.3 Modularity and Louvain algorithm for community detection

Modularity-based methods have been the mainstream among the community detection algorithms. Modularity-based methods consider the community detection problem as the maximization of a quality function, called *modularity*, defined over all the partitions of the network nodes. Newman-Girvan modularity [296, 292] has been a widely adopted classical form for unipartite networks. Given the adjacency matrix $\mathbf{U} = [u_{ii'}]$ in which $u_{ii'}$ is the weight of the edge between nodes v_i and $v_{i'}$, for any partition of nodes in a unipartite network, $\mathcal{P} = (C_1, C_2, \dots, C_K)$, where C_i denotes the index of the community to which v_i belongs (e.g., $C_i = k$ means that the i -th node belongs to the k -th community), the corresponding Newman-Girvan modularity is defined as

$$Q(\mathcal{P}) = \sum_{i=1}^n \sum_{i'=1}^n \left(\frac{u_{ii'}}{u} - \frac{u_i u_{i'}}{u^2} \right) \delta(C_i, C_{i'}). \quad (6.11)$$

In the above expression of modularity, $\delta(C_i, C_{i'}) = 1$ if $C_i = C_{i'}$, i.e., if v_i and $v_{i'}$ belong to the same community and $\delta(C_i, C_{i'}) = 0$ otherwise. $u_i = \sum_{i'=1}^n u_{ii'}$ is the degree of node v_i , i.e., the total weight of edges adjacent to v_i . $u = \sum_{i=1}^n u_i = \sum_{i=1}^n \sum_{i'=1}^n u_{ii'}$ is the total edge weight in the network (with each edge counted twice). Hence the term in parentheses in (6.11) captures the difference between the existing fraction of edges between v_i and $v_{i'}$ and the expected fraction of edges between v_i and $v_{i'}$ when all the edge weights are randomly redistributed with the degree of all nodes fixed, thus measuring the level of interaction between v_i and $v_{i'}$ above a statistical average. The modularity $Q(\mathcal{P})$, as the sum of all such measures inside the communities, characterizes the statistical significance of the communities as given by partition $\mathcal{P}$.

The community detection in bipartite networks is therefore formulated as the maximization of modularity $Q(\mathcal{P})$ with respect to all feasible partitions, and the maximized modularity Q is called the modularity of the network, i.e., $Q = \max_{\mathcal{P}} Q(\mathcal{P})$. The global

maximization of modularity over all the partitions is an NP-hard problem owing to its combinatorial nature [46]. Due to the computational complexity of the global maximization, many approximation algorithms have been proposed (see [114]). The Louvain algorithm or fast unfolding algorithm [39] has been recognized as the most efficient one with an observed linear scalability. The algorithm involves the following steps until the modularity Q reaches a maximal value.

1. Initialize by assigning every node into a separate community.
2. For each node and each community other than the community to which this node belongs, calculate the modularity increase ΔQ if the node is moved from its original community to this new community.
3. Choose the largest $\Delta Q > 0$ among all nodes and all communities and move the corresponding node into the corresponding community.
4. Repeat the steps 2 and 3 for all nodes until no $\Delta Q > 0$ can be found, i.e., the modularity has reached a local maximal value.
5. Agglomerate each community into a node. Define the edge weights between agglomerated nodes as the sum of the edge weights between the communities from which these two nodes are agglomerated. Define a self-edge for every node with a weight equal to the sum of the edge weights within the community from which it is agglomerated.
6. Repeat steps 2 to 5 until no move further increases the modularity value.

For bipartite networks, the modularity assumes a different form than that of unipartite networks [23] due to the absence of any expected edges between the nodes within the same category (in our case, the variables and the constraints). Given the bipartite adjacency matrix $\mathbf{B} = [b_{ij}]$, where b_{ij} is the weight of the edge between nodes v_i and c_j (for unweighted networks all the weights equal to 1), the modularity of any partition $\mathcal{P} = (C_1, C_2, \dots, C_K)$ is expressed as

$$Q = \sum_{i=1}^m \sum_{j=1}^n \left(\frac{b_{ij}}{b} - \frac{b'_i b''_j}{b^2} \right) \delta(C'_i, C''_j) \quad (6.12)$$

in which $b'_i = \sum_{j=1}^n b_{ij}$, $b''_j = \sum_{i=1}^m b_{ij}$ are the degrees of v_i and c_j , respectively. C'_i and C''_j are the community affiliations of nodes v_i and c_j , respectively. $b = \sum_{i=1}^m \sum_{j=1}^n b_{ij}$ is

the total weight in the network. Similar to the unipartite modularity (6.11), the bipartite modularity (6.12) also characterizes the difference between intra-community interactions and its expected level, and hence the statistical significance of the existence of bipartite communities, each of which contains nodes from both categories. The maximization of bipartite modularity can follow the same steps in the Louvain algorithm as described above.

By applying the community detection algorithm to the bipartite and unipartite networks constructed in Subsection 6.2.1, groups of variables (or groups of constraints) are generated such that the variable (constraint) interactions are weak between different groups but strong within each group. In general, these decompositions need not be identical. Community detection in the bipartite network simultaneously partitions variables and constraints, so that the variable pairs in the same group roughly share a corresponding group of constraints. Community detection in the variable (constraint) unipartite network generates groups of variables (constraints) that share more constraints (variables) *in number*, but the constraints (variables) shared by variables (constraints) in the same group can be highly different and need not form groups.

Remark 6.3. *For distributed optimization, it is desirable to have balanced loads for distributed computing units, i.e., similar sizes for the subsystems generated by the decomposition method. However, load balancing can not be guaranteed by community detection. In the case where the subnetworks are highly heterogeneous in size, we can perform successive community detection (further partition) in the subnetworks that are computationally cumbersome, and agglomerate the subnetworks that are too small.*

6.3 Application to Distributed NMPC

We consider a time-invariant system governed by differential algebraic equations (DAEs):

$$\dot{x} = f_0(x, u), \quad 0 = g(x, z), \quad (6.13)$$

in which $x(t) \in \mathbb{R}^{n_x}$, $z(t) \in \mathbb{R}^{n_z}$ are the vectors of differential state variables and algebraic state variables respectively, and $u(t) \in \mathbb{R}^{n_u}$ denotes manipulated inputs. For simplicity, we only consider nominal models without plant-model mismatch, exogenous disturbances and measurement noise here. The MPC of the system (6.13) requires that

at each sampling time the optimization problem (6.14) be solved:

$$\begin{aligned}
\min \quad & J = \sum_{t=0}^{N-1} \Phi(u_{1:n_u}^t, x_{1:n_x}^t) + \Psi(x_{1:n_x}^N) \\
\text{s.t.} \quad & x_{i_x}^t = f_{i_x}(x_{1:n_x}^{t-1}, u_{1:n_u}^{t-1}), \quad 0 = g_{i_z}(x_{1:n_x}^t, z_{1:n_z}^t), \\
& 0 = r_{i_r}(u_{1:n_u}^t, x_{1:n_x}^t, z_{1:n_z}^t), \\
& u_{i_u} \in [\underline{u}_{i_u}, \bar{u}_{i_u}], x_{i_x} \in [\underline{x}_{i_x}, \bar{x}_{i_x}], z_{i_z} \in [\underline{z}_{i_z}, \bar{z}_{i_z}], \\
& i_u = 1, \dots, n_u; i_x = 1, \dots, n_x; \\
& i_z = 1, \dots, n_z; i_r = 1, \dots, n_r; t = 1, \dots, N.
\end{aligned} \tag{6.14}$$

The variables include the manipulated inputs, differential and algebraic states, and outputs at each discretized time $(u_{i_u}^t, x_{i_x}^t, z_{i_z}^t, y_{i_y}^t)$. The constraints include system equations – f_{i_x} (obtained by single step discretization), g_{i_z} (when algebraic equations exist), upper and lower bounds, and any additional coupling constraints (r_{i_r}). The additional constraints are needed for NMPC when, for example, the closed-loop system is required to be stabilized with a prescribed Lyapunov function [234], or when constraints on the manipulated inputs are needed to deal with open-loop instability [327]. Without loss of generality we assume all r_{i_r} are equalities (for inequalities we can appropriately define extra algebraic states with bounds). N represents the number of sampling times in a receding horizon, respectively. The initial states $x_{i_x}^0$ are given by the real-time measurements.

We assume that the stage cost function Φ and the terminal cost function Ψ are separable (e.g., in a classical quadratic form). Then the formulation (6.14) is expressed in the form of (6.1). An illustration of the variable-constraint bipartite network representing the structure of (6.14) is shown in Fig. 6.6 for a simple case. It shows a recurrent pattern throughout the time axis as long as the system is time-invariant, where the algebraic constraints connect variables with the same time index, and the differential constraints connect variables with the neighboring time indices.

In our previous papers [180, 406, 408], the distributed control architecture was generated based on interaction analysis in the system digraph, which is a directed network representation of the relations between process variables in the system. Hence the decomposition was based on process variables, e.g., if u_1 and u_2 are assigned into subsystems 1 and 2 respectively, then the optimization variables $u_1^{0:N-1}$ and $u_2^{0:N-1}$ have to be divided into two subproblems, although u_1^0 through u_1^{N-1} share no constraint and

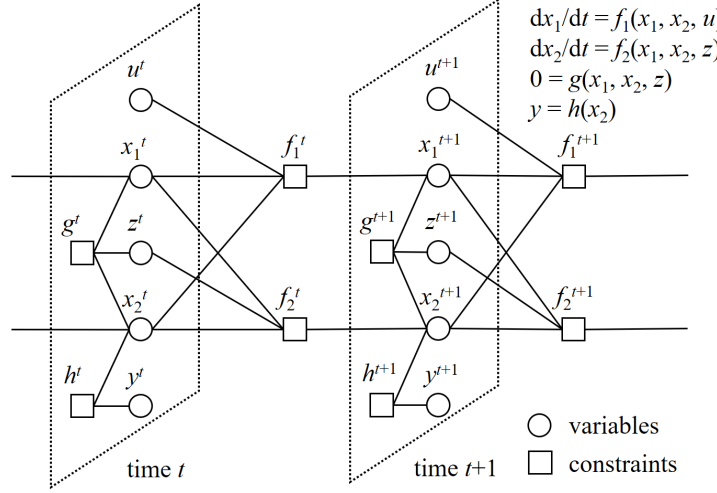


Figure 6.6: An illustration of variable-constraint bipartite network for optimal control problems in NMPC.

u_1^t and u_2^t may interact. Compared to the system digraph-based methods, the variable-constraint network-based method proposed here considers the optimal control problem in its optimization formulation, and detects the communities of optimization variables.

The variable-constraint bipartite network and variable and constraint unipartite networks also provide a more flexible framework than the system digraph, since any other constraints in addition to the system model can be incorporated into the network representation. Moreover, the variable-constraint network can be applied under circumstances where the system digraph can not be well established. For DAE systems which involve algebraic equations, any algebraic constraint can be included by creating a constraint node and variable-constraint edges. For time delay systems where the change of some states is dependent not only on the current states but also on the states of previous times, it suffices to create links between the corresponding constraint node and the state variables with different time superscripts.

6.4 Case Study: Reactor-Separator Process

6.4.1 System description and network decompositions

The reactor-separator process in Fig. 4.1 is a benchmark system for studying distributed model predictive control [234, 397, 421]. The system comprises of two reactors where

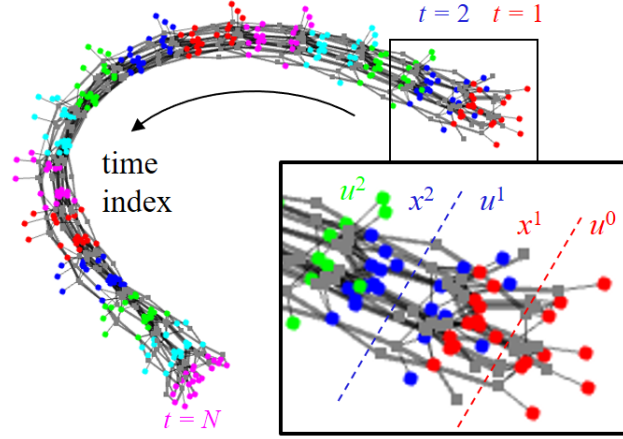


Figure 6.7: Variable-constraint bipartite network for the NMPC of reactor-separator process. The variables are represented by colored circles and the constraints are represented by squares in gray. Different colors represent time indices $t = 1, 2, \dots, N$ for inputs u^{t-1} and states x^t .

reactions in series $A \rightarrow B \rightarrow C$ take place, and a separator producing a product stream withdrawn and a recycle stream. 12 state variables – the concentrations of A and B, temperatures and holdup volumes in three units – describe the system. We use flow rates $F_{f1}, F_{f2}, F_1, F_2, F_3, F_R$ and heat exchange rates Q_1, Q_2, Q_3 as 9 manipulated inputs. The details of the dynamical system model and its NMPC problem are given in Appendix A.

For the optimal control problem in the NMPC of the reactor-separator process, a variable-constraint bipartite network and subsequently a variable unipartite and a constraint unipartite network were constructed. Fig. 6.7 shows the bipartite network, in which the variable nodes are colored according to the time indices (for the inputs u^{t-1} and the states x^t , $t = 1, 2, \dots, N$). The edges connecting the variable nodes with the constraint nodes in grey represent the variable-constraint interactions. The whip-like bipartite network exhibits a recursive pattern due to the time-invariance of the system, with the time index increasing as we traverse from one end to the other end on the whip. For each time index (i.e., each connected variable group in a specific color in Fig. 6.7), the input and state nodes can be seen clearly, as shown in an amplification of the head of the whip in Fig. 6.7.

To depict the network in a more simplified way, we agglomerate the manipulated inputs and system state variables according to their time indices, and count the number

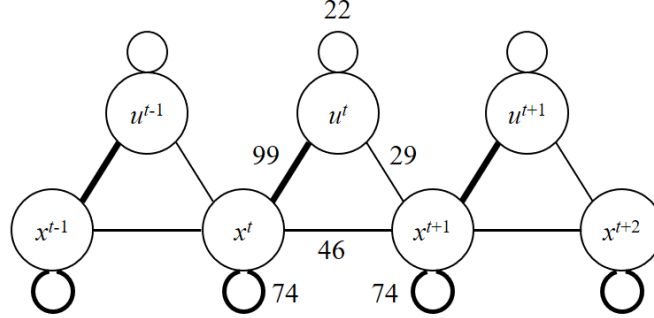


Figure 6.8: Simplified representation of the NMPC of reactor-separator process. Each number represents the total amount of pairwise interactions between two categories of variables.

of pairwise interactions between different types of variables and among themselves (by summing the corresponding edge weights in the unipartite adjacency matrix). This simplification generates a representation of the optimal control problem as a chain of triangles, as shown in Fig. 6.8. To decompose such a network into communities, two of the three edges for some triangles need to be removed. Since the u^t-x^{t+1} and the x^t-x^{t+1} variables have significantly fewer interactions than the u^t-x^t variables, we expect the community detection to generate decompositions by removing the u^t-x^{t+1} and x^t-x^{t+1} edges. In other words, there should be some time indices t such that x^{t+1} is not assigned to the same community as x^t and u^t .

The community detection in the bipartite network and the unipartite network are shown in Fig. 6.9(a) and (b). 8 and 7 communities are generated, respectively. In both cases, the communities are partitioned according to our previous analysis and the decompositions generated appear to be similar. The difference lies in the location where triangles are cut off – the u^t-x^{t+1} and x^t-x^{t+1} edges are removed for $t = 1, 3, 5, 7, 9, 11, 13$ in the bipartite network, and for $t = 2, 4, 6, 8, 10, 12$ in the unipartite network. This pattern, which is closely related to the physical structure of the optimization problem, appears also in the community detection of the constraint unipartite network, the result of which is omitted here for brevity.

For comparison of different decompositions, we apply the community detection in the system digraph [180], which generates 3 communities of process variables, according to the physical topology of the network, and directly extend it to the decomposition of the optimal control problem. That is, for the process variables in each system digraph

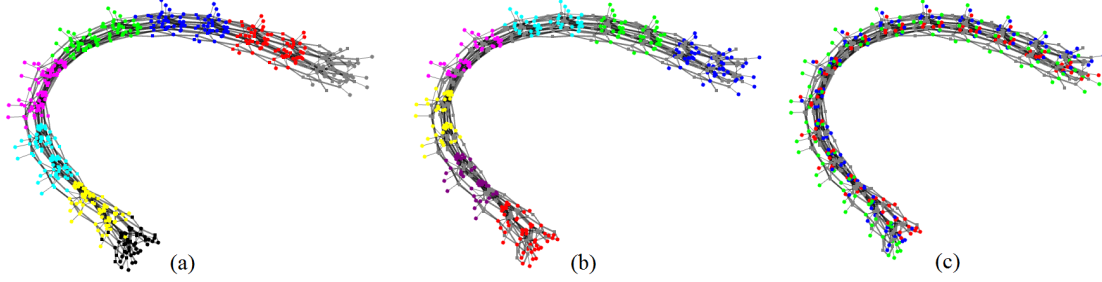


Figure 6.9: Different decompositions in the network representations of the optimal control problem of the reactor-separator process for comparison. (a) Communities in the variable-constraint bipartite network. (b) Communities in the variable unipartite network. (c) Communities in the system digraph. Different colors are used for different communities. Constraints are painted in gray expect for bipartite networks.

community, we assign these process variables with all the time indices $t = 1, 2, \dots, N$ to this community. The decomposition thus obtained is shown in Fig. 6.9(c), where each community (red, blue, green) extends from the head to the tail of the whip. We also consider the decomposition completely based on the time indices of the variables into $N = 15$ subsystems, i.e., each subsystem includes $\{u_{1:9}^{t-1}, x_{1:12}^t\}$ for each $t \in \{1, 2, \dots, 15\}$.

For any decomposition, we characterize its quality by (i) the total *computational time* during the simulation time, and (ii) the control *performance index* (PI), defined as the sum of the stage costs:

$$\text{PI} = \sum_{t=0}^{\infty} \Phi(u^t, x^t) \Delta t. \quad (6.15)$$

Specifically, when the objective function assumes a quadratic form $\Phi(u^t, x^t) = (x^t)^\top Q x^t + (u^t)^\top R u^t$, then PI is the sum of an integrated square error (ISE) term and an integrated square control (ISC) term, namely

$$\text{PI} = \underbrace{\sum_{t=0}^{\infty} (x^t)^\top Q x^t \Delta t}_{\text{ISE}} + \underbrace{\sum_{t=0}^{\infty} (u^t)^\top R u^t \Delta t}_{\text{ISC}}. \quad (6.16)$$

6.4.2 Comparison of decompositions under ADMM

Now we investigate the effect of different decompositions on the distributed NMPC under an ADMM algorithm. The optimal control problem (6.14) for the reactor-separator

process can be expressed in the form of (6.1):

$$\min J(v) \quad \text{s.t.} \quad c(v) = 0, \quad \underline{v} \leq v \leq \bar{v} \quad (6.17)$$

where v is the vector of variables to be optimized containing all the inputs and states within the receding horizon, J is the objective function, $c(v) = [c_1(v), c_2(v), \dots, c_m(v)]^\top$ is the vector of constraints, and $\underline{v}$ and $\bar{v}$ are the element-wise lower bounds and upper bounds for v , respectively. Community detection is performed based on the structure of the constraints $c(v)$. The individual variable bounds $\bar{v}$ and $\underline{v}$ are not used for the decomposition. The distributed optimization of problem (6.17) in a parallel multi-block ADMM formulation with variable decomposition $v^1, v^2, \dots, v^N$ involves the following iterations:

$$\begin{aligned} v^1 &:= \arg \min_{\underline{v}^1 \leq z^1 \leq \bar{x}^1} L_\gamma(z^1, v^2, \dots, v^N; \lambda) \\ v^2 &:= \arg \min_{\underline{v}^2 \leq z^2 \leq \bar{v}^2} L_\gamma(v^1, z^2, \dots, v^N; \lambda) \\ &\dots \\ v^N &:= \arg \min_{\underline{v}^N \leq z^N \leq \bar{x}^N} L_\gamma(v^1, v^2, \dots, z^N; \lambda) \\ \lambda &:= \lambda - \gamma c(v) \end{aligned} \quad (6.18)$$

with the augmented Lagrangian

$$L_\gamma(v^1, v^2, \dots, v^N; \lambda) = J(v) - \lambda^\top c(v) + \frac{\gamma}{2} \|c(v)\|_2^2. \quad (6.19)$$

To decrease the computational burden of the primal variable minimization steps, we approximate the $\arg \min_{z^i}$ with the result obtained by performing only one single step of line search in the gradient direction. The step size is chosen according to the Armijo-Goldstein condition [31], i.e.,

$$\begin{aligned} v^i &:= v^i + \alpha d \\ d &= -\nabla_{v^i} L(v^1, v^2, \dots, v^N; \lambda) \\ \alpha &= \max_{k \in \mathbb{N}} \{ \alpha_0 \delta^k | L_\gamma(v^1, \dots, v^i + \alpha_0 \delta^k d, \dots, v^N; \lambda) \leq \\ &\quad L_\gamma(v^1, \dots, v^i, \dots, v^N; \lambda) - \beta \alpha_0 \delta^k \|d\|_2^2 \} \end{aligned} \quad (6.20)$$

More details of the controller are found in Appendix A.

Table 6.1: Comparison of control and computational performances for different decompositions

Decompositions	ISE	ISC	PI	Time	Iter.
Centralized	4339	818	5157	1496 s	1134
(a) Bipart.	3975	966	4941	779 s	3175
(b) Var. Unipart.	4005	975	4980	653 s	2365
(c) Syst. Digraph	3854	1150	5004	1221 s	2647
(d) Time Index	4078	1047	5125	1445 s	max

¹The “max” stands for the maximum allowed iteration number.

The NMPC based on the decompositions mentioned above along with a centralized ADMM are compared using MATLAB R2017a on a Intel Core i7-2600 CPU with a 16GB RAM, and the results are shown in Table 6.1. Based on the results, we can reach the following conclusions:

1. Compared to centralized control, distributed control under some decompositions (a, b) is capable of decreasing the computational time. Although the distributed algorithms cannot always guarantee a full convergence to the KKT point of the centralized optimal control problem and may be stopped in stagnation, owing to the non-convexity of the problem, it turns out that the decompositions do not deteriorate the overall control performance (PI) compared to that of centralized control.
2. The quality of distributed MPC is not simplistically correlated to the number of subsystems that the original optimization problem is decomposed into. In our case, a fine decomposition into 15 subsystems (d) fails to convergence to an optimum (although the obtained suboptimal solution also gives a stabilizing control). A coarse decomposition into 3 subsystems (c) has longer computational times than the decompositions into 7 and 8 subsystems (a, b). The total computational time is contributed by two factors – the time for a single iteration, and the total iteration number until convergence. In order to find a “good” decomposition, an interaction analysis of the problem structure should be considered.
3. Community detection in the variable unipartite network (b) gives decompositions with better computational performance than that in the variable-constraint bipartite network (a). This is mainly due to the fact that the constraints are dualized in augmented Lagrangian-based optimization algorithms, and hence the interactions

Table 6.2: Comparison of control and computational performances of decomposed and centralized IPOPT

Decompositions	ISE	ISC	PI	Time	Iter.
Centralized	4121	1414	5535	131 s	—
(a) Bipart.	5682	611	6293	25 s	126
(b) Cons. Unipart.	5682	611	6293	35 s	126

between the variables are better captured by the number of terms in which they coexist in the augmented Lagrangian.

4. The optimal decomposition that we have found so far is the one by community detection in the variable unipartite network (b); it brings down the computational time to approximately 44%, and has a control performance slightly better than that of centralized control.

6.4.3 Comparison of decompositions under IPOPT

In the previous subsection, the proposed decomposition method was applied to distributed control under ADMM. However, we also note that ADMM has limitations. For example, its intrinsic slow convergence rate results in a long computation time even for this process system of moderate size.

We also considered a fast nonlinear programming algorithm, IPOPT, for distributed control in the following sense. Through community detection in the variable-constraint bipartite network and constraint unipartite network, we divide the constraints into several groups and find the relevant optimization variables for each group of constraints. In each iteration, we solve in parallel the optimization subproblems, each of which involving a group of constraints and their relevant variables, using IPOPT. If a variable is shared by more than one group of constraints and thus appears in more than one subproblems, each of which generates an optimized value in an iteration, then its value is updated by taking the average. The iterations are terminated according to the same criterion as in ADMM, which is described in Appendix A.

This algorithm is implemented in MATLAB 2017a with OPTI Toolbox v2.27 [70].² The results are shown in Table 6.2. It is seen that the decompositions result in a significant decrease in computation time down to about 19% and 27% of that of centralized

²Available at <https://www.inverseproblem.co.nz/OPTI/index.php>. See also <https://projects.coin-or.org/Ipopt/>.

control, respectively. A small control performance deterioration over the centralized MPC is also observed. The two decompositions give identical control performance due to the similarity in the community structures detected in the underlying network (as we have discussed in Subsection 6.4.1), while the community detection in the bipartite network results in fewer overlapped variables between different groups of constraints, thus reducing the total computational burden.

We note that distributed optimization algorithms can not in general guarantee the convergence to the optimum of the centralized problem, and therefore, we can not guarantee *a priori* the closed-loop stability even when the centralized controller is stabilizing. To this end, after obtaining a good decomposition through community detection, simulations are still needed to examine the control performance of distributed NMPC.

Remark 6.4. [67] proposed a decomposition based on finding an approximate Newtonian direction based on a block diagonal approximation of the KKT matrix. In our case, since the interactions between subsystems are not all weak, it is difficult to do such an approximation. Alternatively, it is in principle possible to use the value of Jacobian elements as edge weights and perform community detection in the resulting Jacobian bipartite network. For this example, we observed that this results in a decomposition that does not lead to convergence of the system states towards the steady state.

Remark 6.5. Although the application of global optimization in optimal control problems has been proposed by [100], global algorithms have not been prevalent in NMPC applications, probably due to the very high computational effort required. However, since global algorithms (e.g., [10, 358]) guarantee the global optimality of the solution, thus offering a potential of further improving the process economics, their application and influence on the NMPC and the resulting computational performances should be explored. The decomposition of global algorithms for distributed optimization, for which Lagrangian-based global algorithms [38] may provide a useful tool, is also an open problem for future studies.

6.5 Conclusions and Discussion

Decomposition techniques are widely used for large-scale and complex optimization problems for a distributed solution. Distributed optimization for NMPC has also been a recent focus of research. This article has proposed community detection as a systematic method of decomposing optimization problems applicable to NMPC.